MongoDB Rework + Extension API — Implementation Plan

1. Context & Motivation

Today the project stores every product as a standalone JSON file in `data/` (~150 files, growing). `database_manager.py` reads/writes those files directly; `scraper.py` threads through them daily. This works but has real limits:

- **No cross-product queries.** `search_products` (database_manager.py:162) globs and linearly scans every file — O(n) per search. Averages, "cheapest today", "biggest price drop this month" are impossible without loading everything.
- **No atomicity.** Two threads writing the same TPNC file race. Currently safe only because each TPNC is processed by exactly one thread, but that's a convention, not a guarantee.
- **Extension coupling.** The Chrome/Firefox extension fetches `https://tesco-price-tracker.gavaller.com/{tpnc}.json` — a *static file URL*. Any storage change breaks this unless we keep serving the same URL shape.
- **No analytics surface.** User explicitly wants averages and aggregations. JSON files can't do that efficiently.

Goal: move canonical storage to MongoDB, preserve the period-folding domain logic exactly, expose a REST API (FastAPI) for the extension and future UI, and keep the existing static-JSON URL working during cutover.

---

2. Proposed Architecture

2.1 Storage: MongoDB, single `products` collection

One document per TPNC, mirroring today's JSON shape 1:1. This is deliberate — not normalized, not time-series — because:

- Period-folding (`_apply_period`, database_manager.py:80) mutates the *last* period in an array. That logic maps cleanly to Mongo array-update operators or a load-modify-save on a single document.
- 16 MB doc limit: each product has ~1 period per price change. At ~3 changes/year × 3 categories × ~100 bytes = trivial. Even at 10 years of daily changes it's well under 1 MB. **Safe.**
- Single-doc reads/writes are atomic in Mongo. The race that exists today disappears.

Document shape (unchanged fields, `_id = tpnc`):

```
{
  "_id": "100002595",
  "tpnc": "100002595",
  "name": "...",
  "unit_of_measure": "each",
  "default_image_url": "...",
  "pack_size_value": null,
  "pack_size_unit": "SNGL",
```

```
  "last_scraped_price": ISODate("2026-03-08T19:57:29Z"),
  "price_history": { "normal": [...], "discount": [...], "clubcard": [...] }
}
```

Indexes:

- `_id` (automatic, used for TPNC lookup).
- `name` text index for `search_products`.
- `last_scraped_price` ascending — makes the "needs scraping today" filter a single indexed query instead of 150 file reads.
- `price_history.normal.end_date` — only if aggregation perf demands it; skip initially, add on measurement.

Rejected alternatives:

- *Separate `price_events` collection with one doc per period.* Cleaner conceptually, but forces a join on every extension read and complicates period-folding (which wants the *last* period cheaply). Not worth it at this scale.
- *Mongo time-series collections.* Optimized for append-only metrics with timestamps. Our data is period-folded (mutates the last entry), which fights time-series semantics. Skip.

2.2 Driver: `pymongo` (sync)

The scraper is threaded with `ThreadPoolExecutor`. `pymongo`'s `MongoClient` is thread-safe and pools connections. Matches the existing synchronous code. `motor` (async) would force a rewrite; `mongoengine`/`beanie` ODMs add schema weight we don't need since we're literally preserving the existing shape.

FastAPI handlers will call `pymongo` directly — FastAPI doesn't require async.

2.3 Deployment: Mongo in `docker-compose.yml`

Add a `mongo` service alongside the existing scraper container. Named volume for `/data/db`. No replica set, no sharding — single node, single developer, single host. Atlas free tier is an option but introduces network latency on every scraper write and an external dependency; self-host wins for this project's scale.

Backup: nightly `mongodump` to the host filesystem via a cron in docker-compose or a sidecar. Simple, restorable.

2.4 API: FastAPI (replace the commented-out Flask stub)

FastAPI gives us Pydantic response models (standardized schema the user asked for), automatic OpenAPI docs (useful when wiring the extension), and better async ergonomics if we later need them. The existing Flask stub (`app.py`) is fully commented — no migration cost.

---

3. API Design

Base path `/api/v1`. All JSON. CORS allow-list: the extension's origin(s) and `localhost` for dev.

Method	Path	Purpose
GET	`/products/{tpnc}`	Full product doc (what extension needs today)
GET	`/products/{tpnc}/history`	Price history, optionally filtered by `category` and date range
GET	`/products/{tpnc}/stats`	**Averages**, min/max, current vs. 30-day-avg — uses Mongo aggregation pipeline
GET	`/products/search?q=...&limit=20`	Text search on name
GET	`/products/cheapest?category=discount&limit=20`	Analytical: biggest current discounts
GET	`/health`	Liveness + Mongo ping
GET	`/{tpnc}.json`	**Compatibility shim** — serves the exact legacy shape so the extension keeps working unchanged

Auth: public read, no key. The Tesco data isn't sensitive and the extension has no login. Rate-limit with `slowapi` (e.g. 60 req/min/IP) to protect against scraping of our scraper.

Pydantic models in `api/schemas.py` — one per response type. This is the "standardized format" the user asked for: the API contract becomes the source of truth for the JSON shape, not the storage layer.

---

4. Migration Strategy — One-Shot Cutover

User choice: one-shot. A single branch lands Mongo, the new `database_manager`, the FastAPI app, the compatibility shim, and the backfill, all together. The `data/` folder is kept read-only on disk as a rollback artifact but is no longer written to after cutover.

Cutover sequence (single deploy window):

1. Stop the scheduler.
2. Bring up Mongo (`docker compose up mongo`).
3. Run `scripts/backfill_mongo.py` — upserts every `data/*.json` into Mongo, verifies doc count == file count, deep-equals 5 random products field-by-field, aborts on any mismatch.
4. Deploy the new scraper + FastAPI container.
5. Start FastAPI; hit `/health` and `/api/v1/products/100002595` to confirm.
6. Point `tesco-price-tracker.gavaller.com/{tpnc}.json` at the FastAPI compatibility shim (DNS/reverse proxy change — whatever currently serves the static files).
7. Re-enable the scheduler.
8. Monitor the first scrape run end-to-end.

Why this is acceptable here despite being riskier than phased:

- Single-developer project, no production users depending on uptime SLAs.
- `data/*.json` is preserved as an immutable snapshot — the backfill reads it, nothing deletes it. Worst case, revert the commit, re-enable the JSON code path, scraper resumes reading/writing files.

- Backfill is idempotent (`upsert` keyed on `_id = tpnc`); safe to re-run if interrupted.

Rollback: `git revert` the cutover commit, redeploy the prior container, delete the Mongo volume if desired. The `data/` folder is still the last known good state because nothing wrote to it after cutover. Maximum data loss window = one daily scrape cycle.

Hard requirement before cutover: the `pytest` suite for `_apply_period` (§9) must pass against *both* backends — the old JSON one and the new Mongo one — with identical results. This is the gate. No green tests, no cutover.

---

5. Upsides

- **Real queries.** Averages, percentiles, "biggest drop this week" become one aggregation pipeline instead of a full directory scan.
- **Atomic writes.** No more implicit assumption that one thread owns one TPNC file.
- **Indexed skip-check.** The daily "who needs scraping?" pre-pass becomes a single indexed query, not 150 file opens.
- **Clean API contract.** Extension and any future UI consume a typed, documented API instead of raw files.
- **Backups become one command** (`mongodump`) instead of a tarball of a directory.

---

6. Downsides & Risks (honest)

- **Operational cost goes from zero to non-zero.** A Mongo process to run, monitor, patch, back up. If the host dies, you restore from `mongodump`, not from a git-tracked `data/` folder.
- **Availability regression during scrape.** Today, if storage fails the scraper crashes but prior data is untouched on disk. If Mongo goes down mid-scrape, writes fail and the retry logic (scraper.py:144) will burn API attempts. Need a circuit-breaker: if Mongo ping fails, abort the run early.
- **Connection pool tuning.** `pymongo` defaults are fine for 2 threads; but if `DEFAULT_THREADS` is later raised, `maxPoolSize` must follow, and the GraphQL 429 rate-limit will still be the real ceiling.
- **Schema drift.** Mongo won't enforce our shape. A bug that writes `"price": "3799"` (string) instead of `3799` (int) will silently corrupt aggregations. Mitigation: Pydantic validation on every write, plus a JSON Schema validator on the collection itself (`$jsonSchema` validator).
- **Index memory.** Small now. If history grows and we add `price_history.normal.end_date` indexes, keep an eye on working-set size.
- **Cold start.** Docker-compose up now has an ordering dependency — scraper must wait for Mongo. Use `depends_on: condition: service_healthy` with a Mongo healthcheck.
- **The extension's current URL is a contract.** Breaking `tesco-price-tracker.gavaller.com/{tpnc}.json` breaks every installed extension instance. The compatibility shim (§3) is non-negotiable until we can ship a new extension version that uses `/api/v1/products/{tpnc}`.

---

7. Cautions / Gotchas

- **Period-folding correctness is load-bearing.** Preserve `_apply_period` (database_manager.py:80) verbatim — just swap the load/save backend. Any "optimization" to use Mongo array operators (`$push`,

`$set` on `price_history.normal.$[last]`) must be gated on a thorough test suite, because the "same-day overwrite" branch (database_manager.py:91-96) is subtle.

- **`datetime` vs ISO string.** Today `last_scraped_price` is an ISO string and `start_date`/`end_date` are `YYYY-MM-DD` strings. Mongo prefers `ISODate`. **Do not convert dates during backfill** — keep them as strings so the period-folding string comparisons still work. Convert only at API boundary if needed.
- **`_id` = `tpnc`.** Use the TPNC as `_id` (string). Saves an index, makes upserts trivial, prevents duplicate TPNCs.
- **Write concern.** Default `w:1` is fine for a single-node deployment. Don't set `w:majority` — there's no majority to wait for.
- **Don't add indexes speculatively.** Each index costs write throughput. Start with `_id`, `last_scraped_price`, and text-on-`name`. Add more only when a specific query is slow.
- **CORS for the extension.** The Chrome extension's origin is `chrome-extension://<id>`. Firefox's is `moz-extension://<uuid>`. Allow both, plus `https://bevasarlas.tesco.hu` if the content script ever fetches directly from a page context.
- **Do not commit Mongo credentials.** `.env` only. Current `.env` is already gitignored — confirm before adding `MONGO_URI`.

---

8. Build Order (within the single cutover branch)

One branch, one PR, one deploy. The work *inside* the branch is still ordered so that each step is independently verifiable before the next:

1. **Infra.** Add `mongo` to `docker-compose.yml` + healthcheck + volume. Bring it up locally; connect with `mongosh`.
2. **Regression tests first.** Write `tests/test_period_folding.py` against the *current* JSON-backed `database_manager`. Must pass — this locks in today's behavior as the spec.
3. **New storage layer.** Refactor `database_manager.py` to hit Mongo. Same public function signatures (`load_product_data`, `save_product_data`, `insert_all_prices`, `get_product`, `get_price_history`, `search_products`, `product_exists`, `init_db`). The `_apply_period` function moves unchanged — only the load/save calls around it change.
4. **Re-run the same test suite against Mongo.** Must pass identically. This is the gate.
5. **Backfill script.** `scripts/backfill_mongo.py`. Verify in a dev Mongo.
6. **Scraper run-state.** Move `run_state.json` logic into a `runs` collection.
7. **FastAPI app.** `api/main.py`, routes, Pydantic schemas, legacy shim at `/{tpnc}.json`, CORS, rate-limit.
8. **Docker-compose glue.** Expose FastAPI port; `depends_on` with healthcheck.
9. **Cutover runbook.** Document the 8-step deploy sequence from §4 in `DEPLOY.md`.

---

9. Verification (gates for the single cutover)

- **Gate 1 — behavior parity.** `pytest tests/test_period_folding.py` passes against both the old JSON backend and the new Mongo backend with byte-identical results. Fixtures cover: same-data-extend, same-day-overwrite, gap-new-entry, first-ever-scrape, all three categories (normal/discount/clubcard).
- **Gate 2 — backfill integrity.** After `scripts/backfill_mongo.py` runs: doc count == file count. A deep-equal diff script walks every `data/*.json` and its Mongo doc — must be zero diffs.

- **Gate 3 — API parity.** `curl http://localhost:8000/100002595.json` (compatibility shim) returns bytes that match `data/100002595.json` exactly. Repeat for 5 products across categories.
- **Gate 4 — extension smoke test.** Install the extension locally, point its fetch URL at the local FastAPI, load a Tesco product page, confirm the chart renders.
- **Gate 5 — aggregation correctness.** For a product with known history, compute the 30-day average manually and compare against `/api/v1/products/{tpnc}/stats`. Must match to the cent.
- **Gate 6 — live scrape.** After cutover, run `python -m scraper --items 100002595 --force` and confirm the Mongo doc updates correctly, period-folding behavior intact.

All six gates must pass before the scheduler is re-enabled.

---

10. Files to Modify / Create

Modify:

- database_manager.py — swap file I/O for Mongo calls; preserve `_apply_period` verbatim; keep the same public function signatures so `scraper.py` needs no changes.
- config.py — add `MONGO_URI`, `MONGO_DB_NAME`, `MONGO_COLLECTION` from env.
- scraper.py — `_load_run_state`/`_save_run_state` move from `run_state.json` to a `runs` collection (one doc per day). No other changes needed thanks to preserved `database_manager` API.
- app.py — replace the commented Flask stub with the FastAPI app (or delete and create `api/main.py`).
- docker-compose.yml — add `mongo` service, healthcheck, `depends_on`, named volume, expose FastAPI port.
- requirements.txt — add `pymongo`, `fastapi`, `uvicorn`, `slowapi`. Remove `flask` (unused).
- Dockerfile — add uvicorn entrypoint option; keep scraper entrypoint.

Create:

- `api/main.py` — FastAPI app factory.
- `api/routes/products.py` — product endpoints.
- `api/routes/stats.py` — aggregation endpoints.
- `api/routes/legacy.py` — `/{tpnc}.json` compatibility shim.
- `api/schemas.py` — Pydantic response models.
- `scripts/backfill_mongo.py` — one-shot migration from `data/*.json`.
- `tests/test_period_folding.py` — regression suite for `_apply_period` (backend-agnostic).
- `tests/test_api.py` — endpoint smoke tests against a test Mongo.

Leave untouched:

- queries.py — no reason to change GraphQL queries.
- scheduler.py — scheduler logic is unaffected.
- data/ — kept on disk through Phase 6 for rollback safety.

Confirmed decisions

- **Mongo host:** local Docker (co-located with scraper in `docker-compose.yml`).
- **Extension URL:** compatibility shim at `/{tpnc}.json` served by FastAPI; new `/api/v1/products/{tpnc}` endpoint shipped alongside for future extension versions.

- **Migration style:** one-shot cutover, gated by a behavior-parity test suite that must pass against both backends before the switch.